

Intraday Tick Simulator

Mathematical Foundations

Philippe Damay

di.simtick, **di.simmarket** and **di.simorder** modules for KDB-X

September 2026

Abstract

This document gives the mathematical framework of a simulator that generates synthetic but realistic intraday market data (trades and quotes) over a calendar of trading days, and works orders against that market.

Trade and quote arrivals are self-exciting Hawkes processes, simulated without approximation. The mid price follows a geometric Brownian motion with optional jumps; its variance accrues with trading activity, and signed order flow moves it through market impact. Quotes are generated first, with spreads in whole ticks, and each trade executes against the quote in force, with buyer- and seller-initiated trades arriving in runs.

Consecutive days are linked by overnight price gaps. Volatility and volume vary from day to day, in quiet and busy periods that last several days, and busy days tend to be volatile ones; any single day can be regenerated on its own.

Orders are split into child orders that execute against the simulated market, either crossing the spread or resting in the queue at the best price, and every order records its full lifecycle of events. An optional step lets the executions move the market, with an impact that partly fades and partly persists.

Contents

1	Introduction	3
2	Random Numbers and Seeds	3
3	Arrivals: Hawkes Processes on Two Clocks	4
3.1	Model	4
3.2	Intraday Profile	4
3.3	Exact Simulation by the Cluster Representation	4
3.4	Jumps Seed Bursts	5
3.5	The Quote Clock	5
4	The Mid: Price Path, Clock and Order-Flow Impact	5
4.1	Geometric Brownian Motion	5
4.2	Jumps as Events	6
4.3	Calendar and Transaction Clocks	6
4.4	Order-Flow Impact	6
5	Quotes	7
5.1	Spread in Ticks	7
5.2	Activity Widens the Spread	7
5.3	Sizes and Imbalance	7
6	Trades Against the Quote in Force	8
6.1	Aggressor Side	8
6.2	Where a Trade Prints	8
6.3	Quantities	8
6.4	Tape Attributes and Auctions	8
7	Days: Overnight Gaps, Regimes and Calendars	8
7.1	Overnight Gap and the Variance Budget	8
7.2	Day-Level Regimes	9
7.3	Calendar	9
7.4	The Output Database	9
8	Orders Against the Tape	10
8.1	Schedule and Sizes	10
8.2	Aggressive and Passive Children	10
8.3	Lifecycle and Tables	11
8.4	Market Impact Across Orders	11
9	Validation	11
10	Limitations	12
11	References	12
A	Configuration Keys of the Orders Group	13
B	Example: One Day of Four US Stocks	14

1 Introduction

A realistic market simulator should reproduce the main features of real intraday data:

- **Trade clustering:** trades arrive in bursts, and quote updates follow them.
- **Intraday pattern:** trading is busy in the first half hour, quiet at midday and busiest before the close; volatility follows the same shape; spreads are widest just after the open and narrow within the first half hour.
- **Price moves:** prices move continuously, with occasional sudden jumps, and a jump is followed by a burst of trading.
- **Order-flow impact:** a buy pushes the price up and a sell pushes it down, and part of the move fades afterwards. Because buys tend to follow buys and sells follow sells, these small moves add up.
- **Quotes and trades:** quotes come first and trades happen against them; the spread is a whole number of ticks, usually one; most trades happen at the bid or the ask, some at the midpoint or slightly inside; trade sizes mix round lots of 100 shares with smaller odd lots; and every trade records its venue, a condition code and a sequence number.
- **Days:** part of each day's price movement happens overnight, between the close and the next open; volatility and volume go through quiet and busy periods lasting several days; and the calendar has holidays and half days.
- **Orders:** a trading algorithm splits a large order into smaller child orders, which either wait in the queue or cross the spread, get re-priced as the market moves, are cancelled or filled, and every step is recorded.

The simulator is made of three KDB-X modules, each building on the one before. **di.simtick** generates one trading day for one instrument; running it for several instruments gives several independent tapes. **di.simmarket** runs it over a calendar of trading days, linking each day to the next. **di.simorder** generates orders and works them against the simulated market.

Each model in the following sections is given with the configuration settings that control it, written like `vol` or `spreadticks`. All random numbers come from seeded generators, so the same configuration always produces the same data.

2 Random Numbers and Seeds

Standard normal variates come from the Box-Muller transform. Let U_1, U_2 be independent uniforms on $(0, 1)$; then

$$R = \sqrt{-2 \ln U_1}, \quad \Theta = 2\pi U_2, \quad Z_1 = R \cos \Theta, \quad Z_2 = R \sin \Theta \quad (1)$$

are independent standard normals. The implementation draws $1 - U$ so that the argument of the logarithm is never 0. Poisson variates with element-wise means λ_i are drawn by inversion: with U_i uniform, X_i is the number of $k \geq 0$ with $F(k; \lambda_i) < U_i$, the cumulative distribution accumulated term by term and truncated at a value beyond which the tail is negligible for the means in use.

A day's stream is seeded once, from the configuration's `seed`. In a multi-day run every date d receives a *regime seed* from the seed and the date alone,

$$s_{\text{regime}}(d) = 1 + (\text{days}(d) + 7919 s) \bmod (2^{31} - 1) \quad (2)$$

shared by every instrument run on that date (the market's day), and from it the instrument's *day seed* and *gap seed*,

$$s_{\text{day}} = 1 + (h(\text{sym}) + 31 s_{\text{regime}}) \bmod (2^{31} - 1), \quad s_{\text{gap}} = 1 + (3 + 17 s_{\text{day}}) \bmod (2^{31} - 1) \quad (3)$$

with h a hash of the ticker's characters. A date's regime innovation is therefore the same in any calendar that contains it, and any day's tape can be regenerated alone from its row of the summary table. The first normal drawn from consecutive seeds was checked to be standard normal, with the squared regime multiplier of Section 7 averaging 1.00 over 26,000 consecutive seeds.

3 Arrivals: Hawkes Processes on Two Clocks

3.1 Model

Trade arrivals exhibit self-excitation: each trade raises the probability of the next.

Definition 1 (Hawkes process). *A univariate Hawkes process $N(t)$ is a counting process with conditional intensity*

$$\lambda(t) = \lambda_0(t) + \sum_{t_i < t} \alpha e^{-\beta(t-t_i)} \quad (4)$$

where $\lambda_0(t)$ is the baseline intensity, $\alpha > 0$ the excitation per event (**alpha**), $\beta > 0$ the decay (**beta**) and $\{t_i\}$ the event times before t .

Proposition 1 (Stationarity). *The process is stationary if and only if the branching ratio $n = \alpha/\beta$ is below 1. n is the expected number of events each event triggers; the mean intensity is $\lambda_0/(1 - n)$.*

3.2 Intraday Profile

The baseline follows the session's profile:

$$\lambda_0(t) = \mu s\left(\frac{t}{T}\right) \quad (5)$$

where μ is **baseintensity**, T the session length in seconds and s a shape function. The shape is a list of positive weights $w_0, \dots, w_{n-1}$, one per equal bin of the session (**profile**, thirteen half hours in the presets), interpolated linearly between bin midpoints and flat beyond the first and last:

$$s(p) = w_i + (x - i)(w_{i+1} - w_i), \quad x = \min\left(n - 1, \max\left(0, np - \frac{1}{2}\right)\right), \quad i = \min(n - 2, \lfloor x \rfloor) \quad (6)$$

A binned profile expresses a flat midday and a spike in the last minutes, which the earlier three-multiplier cosine could not. The shape never exceeds $\max_i w_i$, which the simulation relies on.

3.3 Exact Simulation by the Cluster Representation

Ogata's thinning simulates a Hawkes process by accepting candidates of a Poisson process at a rate that must bound the intensity. With a fixed bound the acceptance probability is capped at 1 in bursts, which under-produces arrivals: 5% at branching ratio 0.4 and threefold at 0.9 in the earlier implementation. The simulator now uses the cluster representation of Hawkes and Oakes (1974), which is exact and needs no bound: immigrants arrive as an inhomogeneous Poisson process of intensity $\lambda_0(t)$, and every event spawns Poisson(n) children at $\text{Exp}(\beta)$ delays,

generation after generation until a generation is empty. The union of all generations is the process.

Algorithm 1: Hawkes process by clusters

```

Input: mu, alpha, beta, session length T, shape s, extra immigrant times E
Output: event times
1: cand <- homogeneous Poisson process on [0,T) at rate mu*max(s)
   (exponential waits drawn in blocks until past T)
2: immigrants <- {c in cand : U_c < s(c/T)/max(s)} union E    # exact thinning
3: parents <- immigrants; events <- immigrants
4: while parents is not empty do
5:   k <- Poisson(|parents| * alpha/beta)                      # children in total
6:   children <- parents[uniform indices of size k] + Exp(beta) delays
7:   children <- {t in children : t < T}
8:   events <- events union children; parents <- children
9: end while
10: return sorted events

```

Line 5 draws the total number of children of a generation and line 6 assigns each child a parent uniformly, which has the same law as $\text{Poisson}(n)$ children per parent and is one vector operation. Counts match the analytic mean $T\mu/(1-n)$ within 1% at every branching ratio tried, up to 0.95, and a day of 500,000 trades takes tens of milliseconds.

3.4 Jumps Seed Bursts

A price jump (Section 4.2) brings a burst of activity that fades over minutes. Each jump at time t_j adds `jumpburst` extra immigrants at exponential delays of mean `jumpburstminutes`, each with its usual cascade, so the burst carries `jumpburst/(1-n)` trades in all. The quote clock receives the same shock scaled by `quotespertrade`.

3.5 The Quote Clock

Quote updates form a second Hawkes process with the same α, β and profile. Its immigrants come from two sources: a background clock at intensity $(1 - \rho_q) q \mu s(t/T)$, and, after each trade, $\text{Poisson}(\rho_q q (1 - n))$ immigrants at $\text{Exp}(\beta)$ delays, where q is `quotespertrade` and ρ_q is `quotetradelink`. The expected number of quote updates per trade is q , and a share ρ_q of them follows the trades, so a burst of trades brings a burst of quotes: with flat profiles the per-second counts of the two clocks correlate at 0.38 against 0 without the link. A quote is placed at the open before all others, so a quote is in force for every trade.

4 The Mid: Price Path, Clock and Order-Flow Impact

4.1 Geometric Brownian Motion

Definition 2 (GBM). *The price S_t satisfies $dS_t = \mu S_t dt + \sigma S_t dW_t$, with drift μ (*drift*), volatility σ (*vol*), both annualized, and W a standard Brownian motion.*

By Itô's lemma, over a sequence of points $t_0 = 0 < t_1 < \dots < t_m$ with steps Δ_j in years,

$$S_{t_i} = S_0 \prod_{j=1}^i \exp \left[\left(\mu - \frac{\sigma^2}{2} \right) \Delta_j + \sigma \sqrt{\Delta_j} \varepsilon_j \right], \quad \varepsilon_j \sim \mathcal{N}(0, 1) \quad (7)$$

The path is sampled on the quote clock, whose first point is the open itself, so S_0 is the price at the open (`startprice`). The mid at a quote is the path there.

4.2 Jumps as Events

Under `pricemodel = jump`, the day's jumps are events: their number is $\text{Poisson}(\text{jumpintensity})$, their times uniform in the session, their sizes lognormal,

$$J_k = \exp(\mu_J + \sigma_J \varepsilon_k), \quad S_t = S_t^{\text{gbm}} \prod_{t_k \leq t} J_k \quad (8)$$

with μ_J and σ_J the keys `jumpmean` and `jumpvol`. A jump is applied to the path from the first quote at or after it, and it seeds the bursts of Section 3.4.

4.3 Calendar and Transaction Clocks

The steps Δ_j depend on the `clock`. Under the calendar clock,

$$\Delta_j = \frac{t_j - t_{j-1}}{N_{\text{days}} T} \quad (9)$$

with N_{days} the trading days per year (`tradingdays`) and T the session in seconds: variance grows with time and realized volatility is flat through the day. Under the transaction clock, the default, every quote update carries the same variance, one trading day's over the day's m updates,

$$\Delta_j = \frac{1}{N_{\text{days}} m} \quad (10)$$

so variance grows with activity: realized volatility follows the intraday profile and rises in bursts. Because the day's variance is normalized by its own number of updates, a busier day does not carry more variance; activity only redistributes it within the day. On the NVDA preset, hourly realized volatility runs from 0.36 at 11:00 to 0.60 in the last hour while the day-level figure stays at the configured 0.45.

4.4 Order-Flow Impact

Signed trades move the mid. The order flow of the day, aggressor signs $\varepsilon_i = \pm 1$ and quantities Q_i (Section 6), is drawn before the quotes, and each trade contributes an impulse in ticks,

$$a_i = \kappa \delta \varepsilon_i \sqrt{\frac{Q_i}{\bar{Q}}} \quad (11)$$

where κ is `impactticks`, δ the tick size and $\bar{Q}$ the mean trade size of the quantity model. A share π (`impactpermanent`) of each impulse stays; the rest halves every h seconds (`impacthalf-life`). The shift in force at a quote time t is

$$D(t) = \sum_{t_i < t} a_i \left[\pi + (1 - \pi) 2^{-(t-t_i)/h} \right] \quad (12)$$

computed as a cumulative sum for the permanent part and a scan of the transient part over the trades, and the mid becomes $S_t + D(t)$. With persistent signs the tape shows price impact and partial reversion: on the NVDA preset the signed markout is 0.34 cents one second after a trade and 0.17 cents after a minute, against 0.007 cents with $\kappa = 0$. Impact adds about 3% to day-level volatility.

5 Quotes

5.1 Spread in Ticks

The spread is a whole number of ticks: one tick plus a Poisson excess whose mean is the mean spread less one tick,

$$\text{spread}_t = \delta (1 + X_t), \quad X_t \sim \text{Poisson}(\max(0, \bar{s} g(t) a(t) - 1)) \quad (13)$$

with $\bar{s}$ the mean spread in ticks through the day (**spreadticks**, 1.15 for the NVDA preset), g the time-of-day multiplier and a the activity multiplier below. The bid is placed so that the spread sits around the mid, on the tick grid:

$$B_t = \delta \left\lfloor \frac{M_t - \text{spread}_t/2}{\delta} + \frac{1}{2} \right\rfloor, \quad A_t = B_t + \text{spread}_t \quad (14)$$

A large cap's spread is widest in the first minutes and tightens within the half hour; it is tightest at the close. The multiplier is the midday level moved toward the open and close levels by exponential decays of e-folding time τ minutes (**spreaddecayminutes**):

$$g(t) = g_{\text{mid}} + (g_{\text{open}} - g_{\text{mid}}) e^{-t/\tau} + (g_{\text{close}} - g_{\text{mid}}) e^{-(T-t)/\tau} \quad (15)$$

with the three levels **spreadopenmult**, **spreadmidmult** and **spreadclosemult**, 2.5, 1 and 0.9 in the presets. On the NVDA preset the spread is one tick 81% of the time, with a midday mean of 1.15 ticks, 1.9 in the first half hour and 1.10 in the last.

5.2 Activity Widens the Spread

Bursts widen spreads. At each quote the number of quote updates in the trailing minute, c_t , is compared with the number the intensity profile expects there, e_t , and the mean spread is multiplied by

$$a(t) = \left(\text{clip} \left[\frac{1 + c_t}{1 + e_t}, \frac{1}{4}, 4 \right] \right)^\gamma \quad (16)$$

with γ the key **spreadactivity** (0.5 in the presets; 0 turns the coupling off). In the minutes after a jump the spread is 1.3 to 2 times its median, and with flat profiles the per-minute spread tracks the per-minute quote count at a correlation of 0.58 against 0.05 without the coupling. Under the transaction clock this also couples spread to volatility, since both follow activity.

5.3 Sizes and Imbalance

Quote sizes are lognormal around **avgquotesize** with log-spread **quotesizevol**, in round lots of 100, and tilted toward the side of the next mid move: with ΔM the sign of the next change of the mid and θ the key **imbalance signal**,

$$\text{bidsize} \propto \exp(\ell z_b + \theta \text{sgn} \Delta M - \frac{1}{2} \ell^2), \quad \text{asksize} \propto \exp(\ell z_a - \theta \text{sgn} \Delta M - \frac{1}{2} \ell^2) \quad (17)$$

so the book leans, weakly, toward what comes next: at $\theta = 0.15$ the size imbalance correlates with the next move at 0.21, a regularity the order book shows, and 0 without the signal. About half of consecutive quotes change only in size, the cancels and replaces of a real feed.

6 Trades Against the Quote in Force

6.1 Aggressor Side

Each trade arrival takes the quote in force at its time. Its aggressor side follows a Markov chain: the first sign is uniform, and each later sign repeats the previous one with probability π_s (**sidepersistence**), so that

$$\text{corr}(\varepsilon_i, \varepsilon_{i-1}) = 2\pi_s - 1 \quad (18)$$

0.4 in the presets, in the range measured on equities. Trades carry the side in an **aggressor** column (B or S), which makes effective spread, realized spread, Lee-Ready classification and markouts well defined.

6.2 Where a Trade Prints

A buyer-initiated trade prints at the ask and a seller-initiated one at the bid, except for a share **midpointshare** that prints at the midpoint and a share **improvementshare** that prints a tenth of a tick inside the touch (price improvement). Prices sit on the tenth-of-a-tick grid. On the presets 80% of trades print at the touch, 12% at the midpoint and 8% inside, the effective spread is 0.5 basis points on NVDA and no trade prints outside its prevailing quote.

6.3 Quantities

Under **qtypemodel** = mixture, a trade's size is a round lot with probability **roundlotshare** (100, 200, 300, 500 or 1000 shares with weights 0.5, 0.2, 0.1, 0.12, 0.08), a block with probability **blockshare** (lognormal, median **blockqty**, log-spread 0.5), and otherwise an irregular lot,

$$Q = \max(1, \lfloor e^X \rfloor), \quad X \sim \mathcal{N}(\ln \bar{Q} - \frac{1}{2}\sigma_Q^2, \sigma_Q^2) \quad (19)$$

with mean $\bar{Q}$ (**avgqty**) and log-spread σ_Q (**qtyvol**). The lognormal and constant models remain. On the NVDA preset 35% of trades are round lots and 55% odd lots, the mean size 153 shares. The mean size of the mixture, which scales the impact of Section 4.4, is $(1 - r - b)\bar{Q} + 260r + b e^{1/8} \text{blockqty}$.

6.4 Tape Attributes and Auctions

Each trade carries a condition (**cond**: R regular, I odd lot, O and C the auction prints), a venue and a sequence number. Midpoint and improved prints are off-exchange with probability 0.95 (venue TRF); prints at the touch are split so that the day's off-exchange share is **offexchangeshare**; lit prints are spread over the MIC codes XNAS, XNYS, ARCX, BATS, EDGX and IEXG by share. One sequence runs across quotes and trades in time order, the opening quote first. The opening and closing auction prints frame the session: at the first and last mid, for **openauctionpct** and **closeauctionpct** of the continuous volume, on the primary listing venue, with no aggressor.

7 Days: Overnight Gaps, Regimes and Calendars

7.1 Overnight Gap and the Variance Budget

Each day after the first opens at the previous close moved by an overnight log return, normal with no drift (the open is a martingale of the close) and variance a share ω (**overnightshare**) of one trading day's, weighted for the calendar days of the gap by κ_g (**gapdayweight**):

$$r_{\text{on}} \sim \mathcal{N}\left(-\frac{1}{2}v, v\right), \quad v = \omega \frac{\sigma^2}{N_{\text{days}}} (1 + \kappa_g(\Delta d - 1)) \quad (20)$$

The intraday simulation runs at $\sigma(1 - \omega)^{1/2}$, so over a one-night gap the close-to-close variance is exactly σ^2/N_{days} : the configured volatility is the close-to-close one, as quoted. A weekend carries more than a night and less than three. Over six months of NYSE days the measured overnight share of close-to-close variance is 29% for $\omega = 0.3$.

7.2 Day-Level Regimes

Days differ, in spells. Two standardized AR(1) states with persistence ϕ (`regimepersistence`), x for volatility and y for volume, are driven by daily shocks with correlation ρ (`regimecorr`):

$$x_d = \phi x_{d-1} + \sqrt{1 - \phi^2} \eta_d, \quad y_d = \phi y_{d-1} + \sqrt{1 - \phi^2} \xi_d, \quad \xi_d = \rho \eta_d + \sqrt{1 - \rho^2} \zeta_d \quad (21)$$

with η_d, ζ_d the two normals drawn from the date's regime seed. The day's multipliers are

$$m_d^{\text{vol}} = \exp(s_v x_d - s_v^2), \quad m_d^{\text{volume}} = \exp(s_q y_d - \frac{1}{2} s_q^2) \quad (22)$$

with s_v and s_q the keys `volregimesd` and `volumeregimesd`. The volume multiplier scales the base intensity and averages 1. The volatility multiplier scales the intraday σ , and since volatility enters a day as variance it is normalized on its square, $\mathbb{E}[(m^{\text{vol}})^2] = 1$, so that the mean daily variance is the configured one: the earlier normalization on the mean inflated close-to-close volatility by $\exp(s_v^2)$. Under the transaction clock a volume regime does not raise volatility (the spread of daily realized volatility is unchanged with the volume regime alone), so the two channels are separate and their correlation is ρ , 0.7 in the default preset. The calendar's own per-day multipliers, closing time and jump intensity multiply on top, for event days and half days.

7.3 Calendar

The calendar is a table of dates with optional closing time, multipliers and jump intensity. A generator produces the NYSE trading days of a range under its rules: weekdays less New Year's Day (not observed on the Friday when it falls on a Saturday), Martin Luther King Jr. Day, Presidents' Day, Good Friday from the Gregorian Easter, Memorial Day, Juneteenth from 2022, Independence Day, Labor Day, Thanksgiving and Christmas, Saturday holidays observed on the Friday and Sunday ones on the Monday, with early closes at 13:00 on the day after Thanksgiving, July 3 and Christmas Eve when they are trading days. 2026 has 251 trading days.

7.4 The Output Database

Several stocks over a calendar are written as a standard date-partitioned kdb+ database, one day of all stocks at a time: for each date in order every stock is simulated for that date, carrying its own previous close into the open, the stocks' tables are joined and the date's partition is written in one step (the trades, the quotes when requested, and a `days` table with one row per stock), sorted by symbol then time with the symbol column parted and enumerated against the root symbol file, every column compressed (128 KB blocks, `zstd` level 3 by default), and the day's tables are dropped. Memory holds one day of all stocks; only each stock's close is carried forward. `days` is written last, so a crash cannot leave a date looking complete: a complete date is skipped on a rerun and an incomplete one rewritten from scratch, and since every date has its own seeds an interrupted run resumed equals a full run exactly. The run itself (every stock's composed configuration, the calendar, the tables, the compression, the module version and the git commit of the code) is stored at the root as the q object `simrun`, so the database is reproducible from that file alone with the same code, and a database built with another configuration is refused rather than mixed.

8 Orders Against the Tape

8.1 Schedule and Sizes

A parent order of quantity Q over a window $[t_s, t_e)$ is sliced into n children (**numfills**) at scheduled times set by the **pacing**: even fractions of the window, the cube of those fractions for a front-loaded order, or the midpoints of n equal intervals for an arrival algorithm. Each time is moved by a uniform jitter of up to **jitter** times half the gap to its neighbours. Sizes are proportional to the market volume in each child's bucket under even pacing, decreasing squares under front-loaded pacing, and, under arrival pacing, the drops of the Almgren-Chriss trajectory

$$X(u) = \frac{\sinh(k(1-u))}{\sinh k}, \quad u \in [0, 1] \quad (23)$$

the share still to trade at time share u of the window, with k the **urgency**, under a participation cap: an interval takes what it wants up to **maxpct** of its volume, $\text{own}/(\text{own} + \text{market})$, carrying shortfalls forward and backfilling unused capacity at the end. Sizes are rounded to integers and the residual placed on the largest child, so they sum to Q exactly.

8.2 Aggressive and Passive Children

Each child is aggressive with probability **spreadcapture** and passive otherwise, and routed to a lit venue by share. An **aggressive child** is a marketable order: after a latency (**latencys**) it takes what the far touch displays at the far touch and the rest one tick beyond, at the same instant, both fills removing liquidity; the book beyond the touch is not modelled and is taken to hold the rest.

A **passive child** is a limit at the near touch in force after the latency, queued behind the size displayed there. Every print by the opposite aggressor at or through the limit takes from the queue first, then from the child, at the child's limit: with c the cumulative quantity of such prints and q the queue at placement, the child's cumulative fill is

$$F = \min(l, \max(0, c - q)) \quad (24)$$

with l its leaves. When the near touch *moves* the algorithm re-pegs, whichever way it went: a replace to the new touch, behind its displayed size, up to **maxreplaces** times.

Which way it moved decides what the move means. A touch that moves **away** leaves the child behind but still inside the market, and a child that has used its replaces simply rests where it is. A touch that moves **through** the limit means the level the child was queued behind has cleared, and the child cannot have survived it: while it rests, a buy *is* the best bid and a sell the best offer, so neither can be traded through. It is therefore filled at its limit at that moment, for at most the size q displayed at that level when it joined,

$$F_{\text{clear}} = \min(l, q), \quad (25)$$

one level of liquidity having gone rather than the whole child, so a child large relative to the touch is not finished by a single clearing. The fill is stamped immediately before the quote that reveals the move, which is the order in which the two happen: the level trades out, then the touch updates to show it gone. A child that has used its replaces stops there rather than resting on, because its limit is now better than anything the market shows and any later fill at it would be at a price no longer offered; its leaves roll on like any unfilled quantity.

At expiry, the next child's send time, what is left is cancelled and rolls into the next child; a final aggressive child before the end of the window completes the order. Fill prices sit on the tick or exactly at the midpoint.

Resting fills are therefore always consistent with the quote in force, and their effective half-spread is negative – a resting order earns the spread and pays for it in adverse selection, which

the markouts show. Fill rates depend on size as they should: on a thin stock a passive child asking for far more than the touch displays fills a small share of itself, and a small one is usually completed by the first level that clears.

A passive child is instead sent to a **dark pool** with probability **darkshare**, the same for every algorithm, and picks its pool by **darkvenueshares**; aggressive children never go dark. A **dark child** is a midpoint peg: after the latency it rests at the midpoint of the quote in force, invisible, never crossing the spread and never re-pegged, since its limit is always the current midpoint. It fills only against opposite-side off-exchange prints at the midpoint, the prints the tape reports as TRF at the mid of the quote in force, a buy against seller-initiated prints and a sell against buyer-initiated ones, taking at most a share **darkfillshare** of each: with P_k the quantities of the qualifying prints in its window, its cumulative fill is $\min(l, \sum_k \lfloor \text{darkfillshare } P_k \rfloor)$. Every dark fill is therefore a print already on the tape, at exactly the midpoint. The tape does not change: the public tape prints off-exchange trades as TRF and cannot identify the pool, while the fill report names it (UBSA, LEVL) and flags the fill D, neither adding nor removing displayed liquidity. Dark liquidity is limited by construction, midpoint off-exchange prints being about 11% of trades, half of them on the right side: on the NVDA fixture order about a tenth of the quantity executes dark, and on a thin stock a dark child often gets nothing and its leaves roll into the next child. With **darkshare** 0 the output is unchanged.

8.3 Lifecycle and Tables

The result is four tables: the parent order with its arrival price (the mid at t_s), filled quantity, average price and status; the children with their type, limit, venue, times, fills, status and replace count; the events, one row per **new**, **ack**, **replace**, **cancel**, **fill** and **done** with quantity, price and leaves; and the executions with venue, liquidity flag and capacity. On the NVDA day of the presets, a patient order (one child in ten aggressive) fills 54 times, 30 of them adding liquidity and 21 dark, with 36 re-pegs; a rushed one (nine in ten) removes liquidity in 28 of its 29 fills and never re-pegs. The patient order's fills are fewer and larger than they were before a resting child was filled as the market reached it, and the same quantity is done. An order flow generator draws many orders per instrument and day from a menu of algorithms, so a multi-day tape carries a population of orders.

8.4 Market Impact Across Orders

The engine prices an order against the market it is given. Impact is a separate step over all the executions of an instrument's day. Under the participation model each execution's impact as a fraction of the price is

$$I_i = \eta \sigma_d p_i^b, \quad p_i = \frac{Q_i}{Q_i + V_i} \quad (26)$$

with σ_d the day's volatility from five-minute mids and V_i the market volume in an interval of the child's length centred on its time; under the square-root law it is $\eta \sigma_d (Q_i/V_{\text{day}})^{1/2}$. The shift in force is the sum of the signed impacts, a share **permanent** of each kept through the day and the rest halving every **halflife**, tapered to zero over **taper** before the close so the next day is unmoved, and rounded to whole cents so that bid and ask move by the same tick. Quotes and prints move by the shift in force at their time, a quote is added at each execution time, and the orders are run again against the moved market.

9 Validation

Each module carries a suite of checks run by a k4unit harness, 48 for the shared configuration, 145 for simtick, 127 for simmarket and 229 for simorder, that hold the invariants (every trade

inside its prevailing quote, spreads whole ticks, quantities conserved, events consistent with fills) and the statistics below.

Property	NVDA preset	Reference for a US large cap
Arrivals vs analytic Hawkes mean, branching 0.3 and 0.9	within 1%	exact
Quote updates per trade	4.0	10 to 30 (set by <code>quotespertrade</code>)
Trades at the touch / midpoint / inside	80% / 12% / 8%	about 80 / 12 / 8
Trades outside the prevailing quote	0	0
Lag-1 autocorrelation of aggressor signs	0.40	0.3 to 0.5
Effective spread	0.49 bp	about 0.5 to 1 bp
Spread of one tick	81%	most of the day
Signed markout at 1 s / 1 min	0.34 / 0.17 cents	a fraction of a cent, reverting
Realized vol, 11:00 hour vs last hour	0.36 vs 0.60	U-shaped
Round lots / odd lots	35% / 55%	large mode at 100, odd lots over half
Off-exchange share	42%	about 40%
Close-to-close vol over two years, configured 0.45	0.449	0.45
Overnight share of daily variance, configured 0.3	0.29	0.3
Correlation of day regimes, configured 0.7	within 0.15	0.6 to 0.8

Statistics reproduced by the implementation on the shipped presets.

10 Limitations

- **Top of the book only:** there is no depth, no queue position beyond the displayed size and no cancels by others ahead of a resting child, so an aggressive child larger than the displayed size walks exactly one tick, and how much of a large resting child a clearing level really fills is bounded by the displayed size rather than measured.
- **The tape does not carry the resting child:** the child is not in the quote data, so what happens to it is inferred from the touch rather than observed. A touch that moves through its limit is taken to mean the level cleared and the child went with it, for that level's displayed size. The inference is what keeps its fills inside the quote in force; what it cannot supply is how much of a large child a single clearing really takes, which needs depth.
- **One-way link between the clocks:** trades seed quote updates; quotes do not excite trades.
- **Orders on the same instrument do not interact** except through the separate impact step.
- **No co-movement between instruments:** three names are simulated independently. The shared per-date regime seed is the hook for a common market day, and a common factor would enter the mid additively on the quote clock, as the order-flow impact does.
- **Special closures** (days of mourning, disasters) are not in the calendar generator.

11 References

- Hawkes, A.G. and Oakes, D. (1974). A cluster process representation of a self-exciting process. *Journal of Applied Probability*, 11(3), 493–503.
- Bacry, E., Mastromatteo, I. and Muzy, J.F. (2015). Hawkes processes in finance. *Market Microstructure and Liquidity*, 1(1), 1550005.
- Almgren, R. and Chriss, N. (2001). Optimal execution of portfolio transactions. *Journal of Risk*, 3(2), 5–39.

- Bouchaud, J.-P., Farmer, J.D. and Lillo, F. (2009). How markets slowly digest changes in supply and demand. In *Handbook of Financial Markets: Dynamics and Evolution*, North-Holland.
- Merton, R.C. (1976). Option pricing when underlying stock returns are discontinuous. *Journal of Financial Economics*, 3, 125–144.
- KDB-X Documentation, Modules. <https://code.kx.com/kdb-x/modules/module-index.html>
- Data Intellect, kdbx-modules. <https://github.com/DataIntellectTech/kdbx-modules>

A Configuration Keys of the Orders Group

The market file's **orders** group, read by `di.simorder`: the venues and their routing shares, the latency, the sweep, the re-peg limit, the jitter, the capacity, the dark pools, the algorithm menu, the defaults of a generated order flow and the impact parameters. The values are those of the shipped US large-cap market. Every key of every module is listed with its layer and group in the generated pages `docs/parameters.md`.

Key	Shipped	Meaning
ordervenues	XNAS ARCX BATS EDGX	lit venues (MIC codes) the child orders are routed to
ordervenueshares	0.4 0.2 0.2 0.2	their routing shares (sum to 1)
latencymms	2	milliseconds from a child's send to its arrival at the market (and half of it to its ack)
sweepticks	1	ticks beyond the touch at which the rest of an aggressive child fills once the displayed size is taken
darkvenues	UBSA LEVL	the dark pools (MPIDs) a passive child can be routed to
darkvenueshares	0.6 0.4	their routing shares among dark children (sum to 1)
darkshare	0.25	probability a passive child is sent to a dark pool instead of a lit venue, between 0 and 1 (0 = no dark routing)
darkfillshare	0.5	the most a dark child takes of an opposite-side off-exchange midpoint print, as a share of it, between 0 and 1
maxreplaces	20	how many times a passive child re-pegs to the near touch when it moves away, before resting where it is
jitter	0.3	random shift of each child's time, as a share of half the gap to its neighbours, between 0 and 1 (0 = exact schedule)
capacity	A	A (agency) or P (principal), the default of the order rows
algos	VWAP IS AGGRESSIVE PASSIVE	the algo menu an order flow draws from (labels)
algotpacings	even arrival frontloaded even	each algo's pacing (even, frontloaded or arrival)
algotspreadcaptures	0.1 0.35 0.9 0	each algo's share of aggressive children, between 0 and 1
algoturgencies	- 2 - -	each algo's urgency (arrival pacing only, null otherwise)
algotmaxpcts	- 0.2 - -	each algo's participation cap (arrival pacing only, null otherwise)
norders	5	orders per instrument and day of a generated flow
accounts	ACC1 ACC2 ACC3	the accounts a generated flow draws from
sizepct	0.005 0.05	lowest and highest order size of a generated flow, as a share of the day's volume
windowminutes	10 60	shortest and longest order window of a generated flow, in minutes
childrenperminute	2	children per minute of window of a generated order (at least 5 children)
orderimpactmodel	participation	impact model of the child executions, participation (p^β) or sqrtlaw
orderimpacteta	0.01	impact coefficient, zero or positive (0 = no impact)
orderimpactbeta	0.5	participation exponent, positive (0.5 is the square-root law)
orderimpacthalflifeseconds	300	seconds over which the transient part of an execution's impact halves
orderimpacttaperminutes	5	minutes before the close over which the impact shift falls linearly to zero
orderimpactpermanent	0.3	share of each execution's impact that stays through the day, between 0 and 1

B Example: One Day of Four US Stocks

This example generates one trading day, 23 September 2026, for four US stocks chosen to span the range of liquidity: a mega cap, a mid cap, a small cap and a micro cap. Together they make costs and market impact visible for transaction cost analysis, and make unusual trading stand out for surveillance: the less liquid the stock, the larger a given order is against its normal

activity.

Stock	Tier	Listing	Price	Market cap	Daily volume (shares)
The Coca-Cola Company (KO)	mega cap	NYSE	\$78	\$335 bn	12–17 m
Hasbro, Inc. (HAS)	mid cap	Nasdaq	\$90	\$12 bn	1.7–2.6 m
Sally Beauty Holdings, Inc. (SBH)	small cap	NYSE	\$16	\$1.5 bn	1.1–1.5 m
Vera Bradley, Inc. (VRA)	micro cap	Nasdaq	\$3.75	\$0.1 bn	0.15–0.5 m

Approximate figures from quotes during 2026.

Calibration. Price, listing and daily share volume follow recent quotes for each stock; volatility, spread, trade sizes and venue shares are estimates for stocks of each type and should be checked against market data before any quantitative use. `tradesperday` and `avgqty` are set together so that the number of trades times the mean trade size of the mixture (Section 6.3) is close to each stock’s daily share volume. `impactticks` follows the square-root law, $\kappa \delta = Y \sigma_{\text{day}} \sqrt{\bar{Q}/V} S_0$ with $Y = 0.3$: small in ticks for the low-priced stocks, but largest in basis points for them, about 3.4 bp per trade for VRA against 0.1 bp for KO.

Venues. Stocks trade most on their own listing exchange, so the lit venue shares depend on the listing: the NYSE split for KO and SBH, the Nasdaq split for HAS and VRA. They apply to the trades printed on exchange; `offexchangeshare` of all trades print off-exchange (venue TRF), and the auctions print on `primaryvenue`. The resulting share of all trades by venue is:

Venue	Coca-Cola	Hasbro	Sally Beauty	Vera Bradley
TRF (off-exchange)	42.0%	40.0%	45.0%	50.0%
XNAS	12.8%	24.0%	12.1%	20.0%
XNYS	16.2%	3.0%	15.4%	2.5%
ARCX	8.1%	9.0%	7.7%	7.5%
BATS	7.0%	7.8%	6.6%	6.5%
EDGX	7.0%	7.8%	6.6%	6.5%
IEXG	2.9%	3.6%	2.8%	3.0%
MEMX	4.1%	4.8%	3.9%	4.0%

Seeds. Each stock gets its own seed. With a shared seed the four would draw the same random numbers and their price paths would move in step.

All parameters. The table lists all 98 configuration keys of the release, with the names used in the code. A value spanning the four columns is shared by all four stocks; it comes from the market file `us_largecap` and the `normal` scenario unless the instrument row overrides it. `baseintensity` is not set but derived by `compose` from `tradesperday` (Section 3.2), here with a mean profile level of 1.004 and no jump bursts. The keys of the orders group configure `di.simorder` (Appendix A describes them) and have no effect on the trades and quotes of this example.

Parameter	The Coca-Cola Company	Hasbro, Inc.	Sally Beauty Holdings, Inc.	Vera Bradley, Inc.
<i>Essential (instrument)</i>				
<code>sym</code>	KO	HAS	SBH	VRA
<code>price</code>	78.00	90.00	16.00	3.75
<code>drift</code>			0.05	
<code>vol</code>	0.16	0.30	0.45	0.65

Parameter	The Coca-Cola Company	Hasbro, Inc.	Sally Beauty Holdings, Inc.	Vera Bradley, Inc.
tradesperday	100000	18000	6000	1300
<i>Session (market)</i>				
openingtime		09:30		
closingtime		16:00		
tradingdays		252		
rngmodel		pseudo		
ticksize		0.01		
printgrid		0.1		
clock		transaction		
<i>Arrivals (market)</i>				
alpha		0.3		
beta		1.0		
profile	1.6 1.2 1.0 0.85 0.75 0.7 0.65 0.7 0.75 0.85 1.0 1.2 1.8			
jumpburst		3000 (no effect: no jumps)		
jumpburstminutes		1.0 (no effect: no jumps)		
quotespertrade		4		
quotetradelink		0.5		
<i>Auctions (market)</i>				
openauctionpct		0.01		
closeauctionpct		0.08		
<i>Trade sizes (market, instrument overrides)</i>				
qtypodel			mixture	
avgqty	40	40	150	200
qtyvol			0.9	
roundlotshare	0.35	0.25	0.35	0.35
roundlots		100 200 300 500 1000		
roundlotweights		0.5 0.2 0.1 0.12 0.08		
blockshare		0.002		
blockqty	10000	5000	5000	5000
blockqtyvol		0.5		
<i>Quotes (market, instrument overrides)</i>				
spreadticks	1.05	2.5	1.5	2
spreadopenmult			2.5	
spreadmidmult			1.0	
spreadclosemult			0.9	
spreaddecayminutes			15	
spreadactivity			0.5	
activitywindowseconds			60	
activityclip			0.25 4	
avgquotesize	2000	300	1000	800
quotesizevol			0.6	
quotelot			100	
imbalanceignal			0.15	
<i>Trades against the quotes (market, instrument overrides)</i>				
sidepersistence			0.7	
midpointshare			0.12	
improvementshare			0.08	
improvementtick			0.1	
offexchangeshare	0.42	0.40	0.45	0.50
offexchangeinside			0.95	
venues	XNAS XNYS ARCX BATS EDGX IEXG MEMX			
venueshars	0.22 0.28 0.14	0.40 0.05 0.15	0.22 0.28 0.14	0.40 0.05 0.15
	0.12 0.12 0.05	0.13 0.13 0.06	0.12 0.12 0.05	0.13 0.13 0.06
	0.07	0.08	0.07	0.08
primaryvenue	XNYS	XNAS	XNYS	XNAS
<i>Order-flow impact (market, instrument overrides)</i>				
impactticks	0.07	0.4	0.2	0.15
impacthalflifeseconds			30	

Parameter	The Coca-Cola Company	Hasbro, Inc.	Sally Beauty Holdings, Inc.	Vera Bradley, Inc.
impactpermanent		0.3		
<i>Scenario: normal</i>				
volmult		1.0		
volumemult		1.0		
spreadmult		1.0		
pricemodel		gbm		
jumpintensity		0		
jumpmean		0		
jumpvol		0		
<i>Scenario: day-to-day keys (used by di.simmarket only)</i>				
overnightshare		0.3		
gapdayweight		0.25		
regimepersistence		0.7		
regimecorr		0.7		
volregimesd		0.3		
volumeregimesd		0.3		
<i>Run</i>				
tradingdate		2026.09.23		
seed	42	43	44	45
generatequotes		true		
<i>Derived by compose</i>				
baseintensity	2.98	0.536	0.179	0.0387
<i>Orders (market; di.simorder only, not used in this example)</i>				
ordervenues		XNAS ARCX BATS EDGX		
ordervenuesshares		0.4 0.2 0.2 0.2		
latencys		2.0		
swepticks		1		
darkvenues		UBSA LEVL		
darkvenuesshares		0.6 0.4		
darkshare		0.25		
darkfillshare		0.5		
maxreplaces		20		
jitter		0.3		
capacity		A		
algos		VWAP IS AGGRESSIVE PASSIVE		
algotacings		even arrival frontloaded even		
algotspreadcaptures		0.1 0.35 0.9 0.0		
algoturgencies		null 2 null null		
algotmaxpcts		null 0.2 null null		
norders		5		
accounts		ACC1 ACC2 ACC3		
sizepct		0.005 0.05		
windowminutes		10 60		
childrenperminute		2		
orderimpactmodel		participation		
orderimpacteta		0.01		
orderimpactbeta		0.5		
orderimpacthalfseconds		300		
orderimpacttaperminutes		5		
orderimpactpermanent		0.3		

Running the example. In q, with the shipped files loaded as `market` and `sc` (the normal scenario), each stock is a dictionary of the keys it sets or overrides, composed and run with its own seed:

```
nas:`XNAS`XNYS`ARCX`BATS`EDGX`IEXG`MEMX!0.40 0.05 0.15 0.13 0.13 0.06 0.08
nys:`XNAS`XNYS`ARCX`BATS`EDGX`IEXG`MEMX!0.22 0.28 0.14 0.12 0.12 0.05 0.07
```

```
k:`sym`primaryvenue`price`drift`vol`tradesperday`spreadticks`avgqty,
  `roundlotshare`blockqty`avgquotesize`offexchangeshare`impactticks
ins:flip k!(`KO`HAS`SBH`VRA; `XNYS`XNAS`XNYS`XNAS;
  78 90 16 3.75; 4#0.05; 0.16 0.30 0.45 0.65;
  100000 18000 6000 1300; 1.05 2.5 1.5 2; 40 40 150 200;
  0.35 0.25 0.35 0.35; 10000 5000 5000 5000; 2000 300 1000 800;
  0.42 0.40 0.45 0.50; 0.07 0.4 0.2 0.15)
ins:update venues:4#enlist key nas,
  venueshares:(value nys;value nas;value nys;value nas) from ins
run1: {[i;s] d:`tradingdate`seed!(2026.09.23;s);
  simtick.run simtick.compose[market;i;sc;d]}
r:run1'[ins;42 43 44 45]
trade:`sym`time xasc raze r@:`trade
quote:`sym`time xasc raze r@:`quote
```